

Difference between JPA, Hibernate and Spring Data JPA

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	Framework/ORM Tool	Framework
Purpose	Defines standards for object-relational mapping	Implements JPA and provides ORM functionality	Simplifies data access using JPA
Implementation	No implementation	Concrete implementation of JPA	Uses JPA implementations such as Hibernate
SQL Handling	Defines APIs only	Generates and executes SQL queries	Further reduces boilerplate code
Transaction Management	Not directly handled	Supports transactions	Automatic transaction management using <code>@Transactional</code>
Coding Effort	Moderate	Less than JDBC	Least amount of code
Example	<code>EntityManager.persist()</code>	<code>session.save()</code>	<code>repository.save()</code>

JPA (Java Persistence API)

- JPA is a Java specification (JSR 338) for managing relational data in Java applications.
- It provides interfaces and annotations such as `@Entity`, `@Table`, `@Id`, and `EntityManager`.
- JPA itself does not provide any implementation.
- Hibernate is one of the most popular implementations of JPA.

Hibernate

- Hibernate is an Object Relational Mapping (ORM) framework.
 - It implements JPA specifications.
 - It converts Java objects into database records and vice versa.
 - It handles SQL generation automatically.
-
- **JPA** defines the persistence standard.
 - **Hibernate** implements the JPA specification and performs ORM operations.
 - **Spring Data JPA** sits on top of JPA/Hibernate and reduces coding effort by providing repositories and automatic query generation.

Hibernate and Spring Data JPA

```
private static void testAddEmployee() {

    LOGGER.info("Start");

    Employee employee = new Employee();

    employee.setId(1);
    employee.setName("John");

    employeeService.addEmployee(employee);

    LOGGER.info("Employee Added Successfully");

    LOGGER.info("End");
}
```

Spring Data JPA

EmployeeRepository.java

```
package com.cognizant.orm_learn.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.cognizant.orm_learn.model.Employee;

@Repository
public interface EmployeeRepository
    extends JpaRepository<Employee, Integer> {

}
```

EmployeeService.java

```
package com.cognizant.orm_learn.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.cognizant.orm_learn.model.Employee;
import com.cognizant.orm_learn.repository.EmployeeRepository;

@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;

    @Transactional
    public void addEmployee(Employee employee) {
        employeeRepository.save(employee);
    }

}
```

OUTPUT:

```
initiateService 59 HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA pla
buildNativeEntityManagerFactory 447 Initialized JPA EntityManagerFactory for persistence unit 'default'
startServer 59 LiveReload server is running on port 35729
logStarted 59 Started OrmLearnApplication in 3.6 seconds (process running for 5.88)
main 27 Inside main
testAddEmployee 34 Start
logStatement 135 select e1_0.id,e1_0.name from employee e1_0 where e1_0.id=?
logStatement 135 insert into employee (name,id) values (?,?)
testAddEmployee 43 Employee Added Successfully
testAddEmployee 45 End
destroy 660 Closing JPA EntityManagerFactory for persistence unit 'default'
close 349 HikariPool-1 - Shutdown initiated...
close 351 HikariPool-1 - Shutdown completed.
```